Project Technical Evaluation: owaspscan

Comprehensive Assessment of Research Gaps, System Architecture, and Implementation Progress

Target Repository: owaspscan (Dual SAST & DAST Engine)

Compliance Standard: OWASP Top 10:2021 Taxonomy

Runtime Footprint: Python 3.9+ (Zero 3rd-party dependencies)

Evaluation Date & Version: October 2026 | v0.1.0 Architecture Specification

---

Executive Summary

owaspscan is an ultra-lightweight, zero-external-dependency security CLI designed to bridge static source analysis (SAST) and live web transport auditing (DAST) into a unified developer workflow. It enforces security checks aligned with the OWASP Top 10 (2021) standard, featuring line-by-line regex matching, Python AST inspection, requirements manifest auditing, and baseline-aware HTTP/TLS dynamic probing. This evaluation provides an in-depth breakdown across three core dimensions: the academic and industrial **Research Gaps** separating this prototype from modern state-of-the-art tools, the internal **System Architecture** and data pipelines, and the current **Implementation Progress** across each OWASP vulnerability category.

1. Research Gap Analysis

While owaspscan achieves high speed and independence from external packages, it currently operates on syntactic heuristics rather than semantic flow formalisms. Bridging the gap to enterprise-grade security analysis requires addressing several theoretical and architectural limitations.

1.1 Static Analysis (SAST) Formalisms & Gaps

- **Source-to-Sink Taint & Dataflow Tracking:** The scanner relies on single-line regular expressions (e.g. `open(...request.args...)`). In real applications, untrusted user input (sources) propagates across variables, helper utilities, and object properties before reaching dangerous operations (sinks). The lack of taint analysis causes both false negatives (missed multi-line flows) and false positives (flagging sanitized inputs).
- **Call Graph & Control Flow Graph (CFG) Construction:** The AST inspector currently performs shallow walks on function decorators and bodies looking for keyword strings (e.g. `login_required`, `current_user`). It lacks an interprocedural call graph, alias analysis, or Static Single Assignment (SSA) intermediate representation. It cannot recognize framework-level middleware, blueprint auth guards, or dependency-injected security dependencies (such as FastAPI Depends).
- **Polyglot Syntax Tree Parsing:** While regex rules support multi-language file extensions (.js, .ts, .php, .java, .go, .rb), AST parsing is restricted exclusively to Python via the standard `ast` library. Non-Python languages receive no semantic structure validation. Adopting Tree-sitter or concrete syntax trees (CST) is necessary for true polyglot analysis.
- **Software Composition Analysis (SCA) & Advisory Feeds:** The manifest scanner merely flags unpinned dependency versions (missing `==`). It lacks dependency tree resolution, transitive dependency graph traversal, lockfile parsing (Pipfile.lock, poetry.lock), and queries against vulnerability advisories (OSV, CVE, NVD, or PyPI Advisory Database via pip-audit).

- **OWASP A04 Insecure Design & Threat Modeling Formalisms:** A04 flaws (e.g. lack of rate limiting, insecure credential recovery, business logic flaws) cannot be detected by pattern matching. Emerging research relies on automated STRIDE threat modeling, formal specifications, and LLM-assisted design reasoning to uncover architectural security weaknesses.

1.2 Dynamic Analysis (DAST) Formalisms & Gaps

- **Automated Crawling & Attack Surface Mapping:** `webscan` audits only explicitly provided URLs and query parameters. It lacks a dynamic spider or headless browser (such as Playwright/Chromium) to explore client-rendered Single Page Applications (SPAs), discover hidden links, or trigger DOM events and form submissions.
- **Session State & Multi-Role Authentication Testing:** The dynamic scanner operates purely unauthenticated. It cannot maintain stateful sessions, handle CSRF tokens, or perform privilege escalation testing (BOLA / IDOR) across multi-tenant authorization boundaries.
- **Advanced Injection Oracles & Exploit Verification:** The SQLi probe relies exclusively on error-based pattern matching triggered by single quotes. It cannot detect Blind SQLi (time-based SLEEP or boolean differential queries) or Second-Order SQLi. Similarly, XSS checking is limited to verbatim reflection of GET parameters, missing DOM-based XSS, context-sensitive escaping, and POST payloads.
- **Protocol & Cipher Suite Depth:** Currently limited to HTTP/1.1 via `http.client` and basic TLS handshake metadata. It lacks deeper auditing for HTTP/2, HTTP/3, WebSocket vulnerabilities, or fine-grained TLS cipher suite negotiation weaknesses.

1.3 Comparative State-of-the-Art Benchmark

Analysis Dimension	SOTA Industry Tools (CodeQL, Semgrep, ZAP)	owaspscan Current State	Recommended Evolution
SAST Depth	Interprocedural dataflow & taint propagation graphs	Line regex + shallow AST checks	Tree-sitter CSTs + source/sink taint rules
Interoperability	OASIS SARIF 2.1.0, GitHub Security Alerts	Custom text summary & JSON output	Implement SARIF serializer in reporters.py
SCA Intelligence	Lockfile resolution + real-time CVE/OSV matching	Manifest regex checking for unpinned versions	Query OSV/PyPI advisory API in scanner.py
DAST Attack Surface	Headless browser DOM spidering & event triggering	Static URL parsing & query extraction	Integrate lightweight headless crawler
Injection Oracles	Time/boolean blind SQLi, DOM XSS, OAST callbacks	Single-quote regex error match & reflection	Introduce differential timing & OAST probes

2. System Architecture

owaspscan follows a modular, decoupled architecture centered around clean separation of data models, rule declarations, execution engines, and reporting adapters.

2.1 Core Modules & Responsibilities

- **cli.py (Entry Point):** Manages argument parsing using `argparse`, subcommand dispatch (`scan`, `web`, `rules`), severity filtering, and CI failure threshold gating (`--fail-on`).
- **models.py (Data Domain):** Defines `Severity` IntEnum (LOW, MEDIUM, HIGH, CRITICAL), the `Finding` normalized dataclass, and the OWASP Top 10:2021 mapping dictionary.
- **scanner.py (Static Engine):** Implements recursive file discovery (`iter_files`) with directory pruning (.git, node_modules, venvs, tests) and file size bounds (MAX_BYTES = 1MB). Orchestrates regex scanning, AST scanning, and requirements manifest parsing.
- **rules/ (Rule Catalog):** Separates rule definitions from traversal logic. `regex_rules.py` defines 14 compiled RegexRule frozen dataclasses. `ast_rules.py` implements Python AST visitors for route IDOR detection (OWS-A01-002) and swallowed exceptions (OWS-A09-002).
- **webscan.py (Dynamic Engine):** Performs HTTP(S) network requests via `http.client` following up to 6 redirects. Gathers TLS certificates and versions. Runs passive audits (headers, cookies, transport) and active probes (soft-404 baseline diffing and injection reflection).
- **reporters.py (Presentation Layer):** Outputs formatted terminal text with severity tags and remediation guidance, or structured JSON for CI/CD pipeline integration.

2.2 Static Analysis Pipeline Flow

> 1. **Ingestion:** `scan_paths()` accepts CLI path arguments and invokes `iter_files()`.
> 2. **Pruning:** Directories matching `SKIP_DIRS` (.git, node_modules, venv, tests) and files > 1 MB are ignored.
> 3. **Multi-path Analysis:**
>   - *Manifest Audit:* `requirements*.txt` files are scanned for unpinned packages.
>   - *Regex Inspection:* Lines are matched against extension-filtered RegexRules. Comments and lines tagged with `# noscan` are skipped.
>   - *AST Parsing:* `.py` files are parsed into an AST and analyzed with `AST_CHECKS`.
> 4. **Consolidation:** Findings are deduplicated by `(rule_id, path, line)` and sorted in descending severity.
> 5. **Output & CI Gate:** Results are filtered by `--min-severity` and formatted. If any finding exceeds `--fail-on`, exit code 1 is emitted.

2.3 Dynamic Analysis Pipeline Flow

> 1. **Permission Guard:** Verifies `--i-have-permission` flag before sending network traffic.
> 2. **Fetch Chain:** Connects via `http.client`, follows redirects up to 6 hops, inspects TLS certificate validity and TLS protocol version.
> 3. **Passive Auditing:** Evaluates response headers for CSP, HSTS, X-Frame-Options, and version disclosure. Audits Set-Cookie headers for HttpOnly, Secure, and SameSite attributes.
> 4. **Soft-404 Baseline Probing:** Requests a random nonexistent endpoint to establish baseline response size and status code, preventing false positives on catch-all servers.

5. **Sensitive Path Discovery:** Sequentially tests 13 critical endpoints (.git/HEAD, .env, .htpasswd) paced by `--delay`.
6. **Parameter Probing:** Injects unique test tokens into GET query parameters to test unescaped HTML reflection (XSS) and database error triggers (SQLi).

3. Implementation Progress

owaspscan implements substantial coverage across 9 of the 10 OWASP Top 10 (2021) categories through its combined SAST and DAST engines. Category A04 (Insecure Design) is intentionally omitted as it requires architectural threat modeling rather than pattern matching.

3.1 OWASP Top 10:2021 Coverage Matrix

OWASP Category	SAST Rule IDs	DAST Rule IDs	Coverage
A01: Broken Access Control	OWS-A01-001 (Path Traversal) OWS-A01-002 (Route Missing Auth)	OWS-WEB-A01-001 (Exposed Sensitive Paths)	HIGH
A02: Cryptographic Failures	OWS-A02-001 (MD5/SHA1) OWS-A02-002 (Weak RNG) OWS-A02-003 (verify=False) OWS-A02-004 (Hard-coded Secrets)	OWS-WEB-A02-001 (Plain HTTP) OWS-WEB-A02-002 (TLS Failed) OWS-WEB-A02-003 (TLS Expired) OWS-WEB-A02-004 (Weak TLS)	HIGH
A03: Injection	OWS-A03-001 (SQL String Concat) OWS-A03-002 (Shell Execution) OWS-A03-003 (eval/exec) OWS-A03-004 (SSTI / Template)	OWS-WEB-A03-001 (Reflected XSS) OWS-WEB-A03-002 (Single-Quote SQL Error)	HIGH
A04: Insecure Design	*None (Requires Threat Modeling)*	*None*	NONE
A05: Security Misconfiguration	OWS-A05-001 (debug=True) OWS-A05-002 (Wildcard CORS)	OWS-WEB-A05-001 (Missing Headers) OWS-WEB-A05-002 (Version Leak) OWS-WEB-ERR (Unreachable)	HIGH
A06: Vulnerable Components	OWS-A06-001 (Unpinned Dependency)	*None*	LOW
A07: Identification & Auth	OWS-A07-001 (JWT Verify False / None Alg)	OWS-WEB-A07-001 (Missing HttpOnly) OWS-WEB-A07-002 (Missing Secure) OWS-WEB-A07-003 (Missing SameSite)	MEDIUM
A08: Software & Data Integrity	OWS-A08-001 (Unsafe Deserialization) OWS-A08-002 (Unsafe YAML Load)	*None*	MEDIUM
A09: Logging & Monitoring	OWS-A09-001 (Sensitive Data in Logs) OWS-A09-002 (Swallowed Exceptions)	*None*	MEDIUM
A10: SSRF	OWS-A10-001 (Outbound User Request)	*None*	MEDIUM

3.2 Codebase Health & Verification Metrics

- **Architecture Footprint:** ~650 lines of clean, modular Python standard library code across 8 modules.
- **External Dependencies:** 0 runtime dependencies in `pyproject.toml` (builds with setuptools>=61).
- **Test Suite Performance:** 18 automated unit tests in `tests/` executing in 0.57s with 100% pass rate. Includes mock HTTP server testing headers, cookies, soft-404 handling, and query reflection.
- **Sample Benchmark:** Executed against `samples/vulnerable_app.py` and `samples/requirements.txt`: Successfully identified 17 vulnerabilities (2 Critical, 9 High, 3 Medium, 3 Low).
- **Operational Tooling:** Fully functional CLI commands ('scan', 'web', 'rules'), line suppression (`# noscan`), and CI gate thresholds (`--fail-on`).